

Projet - Agent intelligent pour le jeu Quoridor

Récapitulatif des échéances

Soumission de votre V1 : 20 septembre 2026 sur Abyss

Soumission de votre V2 : 4 octobre sur Abyss

Soumission de votre V3 : 25 octobre sur Abyss

Soumission de votre V4 : 8 novembre sur Abyss

Soumission de votre V5 : 22 novembre sur Abyss

Agent pour le concours : 15 novembre 2026 sur Moodle.

Code et Rapport à remettre le 3 décembre 2026 sur Moodle.

Consignes (en bref)

- Le projet doit être réalisé par groupe de 2 au maximum. Il est fortement recommandé d'être 2.
- Lors de votre soumission, donnez votre rapport au format **PDF** (matricule1_matricule2_Projet.pdf)
- Lors de votre soumission, votre code `my_player.py` et ses dépendances (**requirements.txt**) doivent se trouver à la **racine du fichier de rendu** compressé au format **ZIP** (matricule1_matricule2_Projet.zip). Plus de détails sont fournis dans le sujet, notamment concernant les dépendances et le fichier **requirements.txt**.
- Indiquez vos noms et matricules dans le fichier PDF et en commentaires en haut du fichier de code soumis.
- Toutes les consignes générales du cours (interdiction de plagiat, restrictions liées aux SIAGs, etc.) s'appliquent à ce projet.
- Il est permis (et encouragé) de discuter de vos pistes de solution avec les autres groupes. Par contre, il est formellement interdit de reprendre le code d'un autre groupe ou de copier un code déjà existant (StackOverflow ou autre). Tout cas de plagiat sera sanctionné par la note minimale.

1 Introduction

Munis de vos nouvelles expertises en intelligence artificielle, vous devez maintenant implémenter un agent capable de jouer efficacement au jeu de plateau *Quoridor*. *Quoridor* est un jeu stratégique où deux joueurs (blanc et noir) s'affrontent sur un plateau de 9x9 pour être le premier à faire parvenir son pion représentatif de l'autre côté du plateau, en effectuant des déplacements et en plaçant des murs pour gêner l'adversaire. Dans le cadre du cours, un plateau de 14x14 sera utilisé, bien que d'autres tailles soient disponibles.

Préparation du jeu Les joueurs se placent face à face, l'un représenté par le pion blanc, l'autre par le pion noir. Une représentation du plateau est présentée dans la Figure 1.

Déroulement d'une partie À chaque tour, un joueur réalise une action parmi les deux suivantes :

1. **Déplacer son pion sur une case adjacente** : Les cases adjacentes à une autre case sont situées en haut, en bas, à gauche et à droite de cette case (pas de déplacement en diagonale, sauf exception -

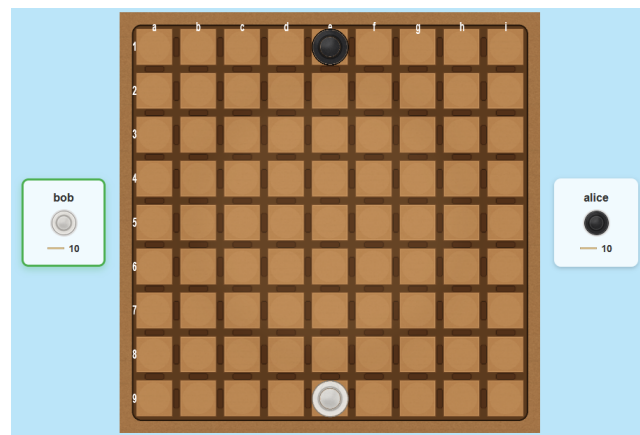


FIGURE 1 – Plateau de départ

voir plus bas). Un joueur ne peut réaliser un déplacement si celui-ci l'amène hors du plateau, ou si la case de départ et celle d'arrivée sont séparées par un **mur** (cf. action suivante).

2. **Placer un mur** : Chaque joueur reçoit, au début de la partie, 10 murs de 2 cases de long. Ils sont placés horizontalement ou verticalement entre deux paires de tuiles adjacentes afin de les séparer. Ils peuvent être placés n'importe où sur le plateau tant qu'ils respectent deux conditions :

- Deux murs ne peuvent pas se chevaucher ou s'intersecter.
- Il est interdit de placer un mur qui empêcherait l'un des joueurs de se rendre de l'autre côté.

Une fois posé, un mur ne peut plus être déplacé pour le reste de la partie. Lorsqu'un joueur a posé tous ses murs, il ne peut plus effectuer cette action.

La partie s'arrête lorsqu'un des joueurs a réussi à amener son pion sur la rangée opposée à sa rangée de départ. Un exemple de plateau pouvant être vu durant une partie est fourni dans la figure 2.

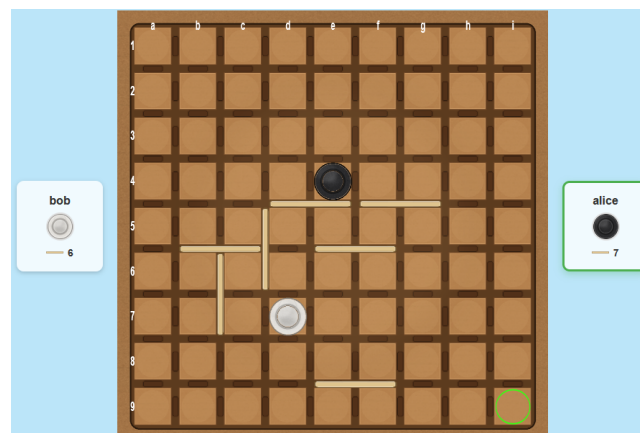


FIGURE 2 – Exemple de plateau au cours d'une partie

Déplacements spéciaux Si un joueur déplace son pion sur une case déjà occupée par celui de son adversaire, il effectue un déplacement spécial.

1. Si un deuxième déplacement dans la même direction est permis, le joueur l'effectue. Son pion a alors "sauté" par-dessus celui de son adversaire (voir Figure 3).

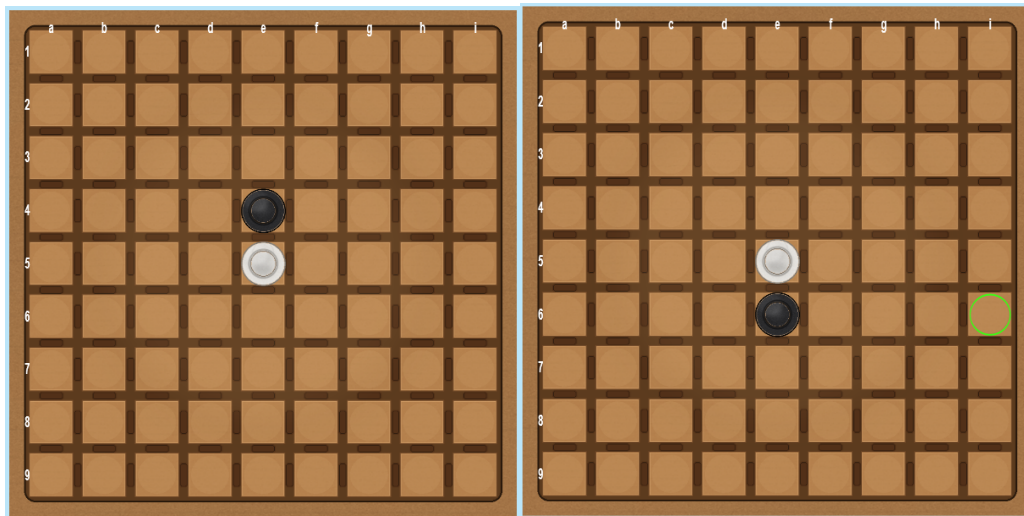


FIGURE 3 – Exemple de saut : le pion noir a sauté par-dessus le pion blanc

- Si le saut n'est pas possible, le joueur effectue un second déplacement perpendiculaire au premier. Ce second déplacement ne peut toujours pas le faire sortir du plateau ni le faire traverser un mur. Un exemple de saut en diagonale est fourni (voir Figure 4)

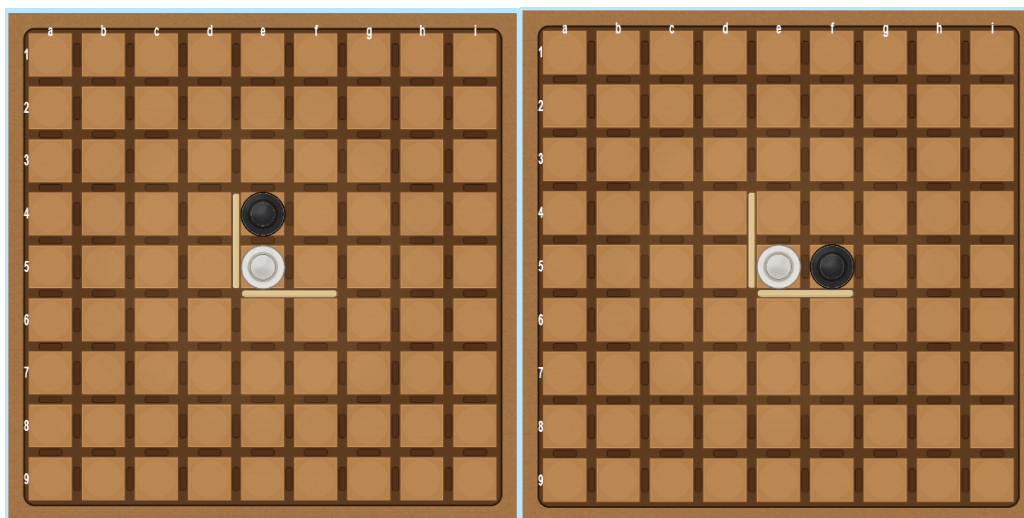


FIGURE 4 – Exemple de saut : le pion noir a réalisé un déplacement en diagonale de e4 vers f5 autour du pion blanc. En raison du mur séparant les cases e5 et d5, un déplacement vers d5 n'aurait pas été possible

- Si ni le saut ni le saut en diagonale ne sont possibles, car des murs ou les bords du plateau les empêchent, un joueur ne peut pas déplacer son pion sur une case déjà occupée par son adversaire.

Vainqueur Le joueur vainqueur est celui qui parvient le premier à amener son pion sur la rangée opposée à sa rangée de départ. Un exemple de plateau en fin de partie est présenté dans la Figure 5.

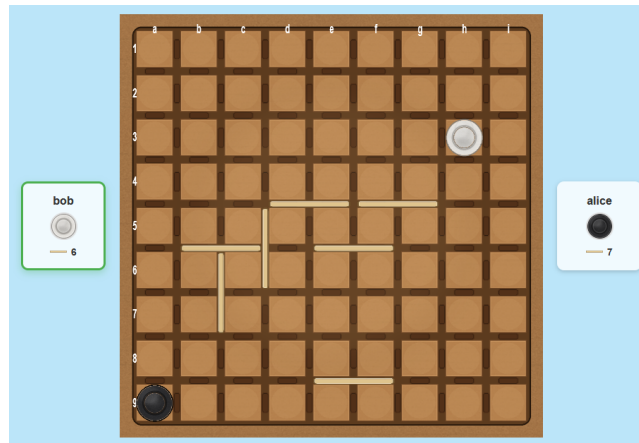


FIGURE 5 – Exemple de plateau en fin de partie, avec une victoire du joueur noir

⚠ Nous vous conseillons de lire la liste complète et succincte des règles (toutes citées ci-dessus) disponibles à l'adresse [suivante](#) et de jouer vous même pour maîtriser les règles. En cas de problème, n'hésitez pas à communiquer avec votre chargé de laboratoire via Discord ou lors des séances de laboratoire.

2 Consignes du projet

Vous êtes libres d'utiliser les méthodes et les bibliothèques de votre choix. Les seules contraintes sont le langage de programmation (**vous devez implémenter votre agent en Python 3.11.x ou supérieur**) et la limite de temps. **Il n'est pas permis d'utiliser d'autres langages de programmation de façon détournée (p.ex., via `pybind11` pour appeler du code en C).** Il est cependant autorisé d'utiliser certaines bibliothèques qui ne sont pas implémentées en Python (p.ex., `numPy`). En cas de doute sur l'utilisation d'une bibliothèque, vous pouvez vérifier auprès de votre chargé de laboratoire. Chaque agent dispose d'un budget de temps de **15 minutes** à répartir sur l'ensemble de ses actions. Ainsi, outre une stratégie standard consistant à allouer le même temps d'exécution à chaque action, il est tout à fait permis à un agent de consacrer 10 minutes à effectuer son premier coup et d'utiliser les 5 minutes restantes pour réaliser les autres coups. Afin de garantir une certaine équité entre les équipes, l'utilisation de **GPU** et du **multithreading** est interdite et entraînera une disqualification. En raison des limites physiques du serveur, votre agent ne doit pas allouer plus de **4 Gb de RAM** (largement suffisant) et **100 Mb de mémoire disque** après compression. En cas d'action invalide, d'utilisation de mémoire trop importante ou de ressources non autorisées, **votre agent perdra inévitablement la partie**. Veuillez donc vous assurer du comportement de votre agent avant toute soumission sur Abyss ou avant le rendu du projet.

3 Ressources fournies

Une version en Python du jeu Quoridor est fournie sur Moodle. Elle implémente l'ensemble des caractéristiques du jeu et permet à un utilisateur de jouer contre un agent via une interface graphique ou de faire s'affronter deux agents intelligents. Les fichiers fournis sont :

- `actions_quoridor.py` : contient le code représentant les actions possibles. **Vous n'avez pas à modifier ce fichier.**
- `board_quoridor.py` : contient le code pour représenter le plateau de jeu. **Vous n'avez pas à modifier ce fichier.**

- `main_quoridor.py` : contient l'ensemble des fonctions permettant de lancer une partie, de l'enregistrer et de la visionner via une interface graphique. Les détails concernant les commandes permettant d'interagir avec l'environnement (notamment pour lancer une partie) sont fournis ci-dessous. **Vous n'avez pas à modifier ce fichier.**
- `master_quoridor.py` : contient le code qui détermine le gagnant du jeu. **Vous n'avez pas à modifier ce fichier.**
- `player_quoridor.py` : contient le code implémentant la classe `PlayerQuoridor` dont votre agent va hériter. **Vous n'avez pas à modifier ce fichier.**
- `game_state_quoridor.py` : implémente toute la logique du jeu Quoridor à l'aide de différentes fonctions. **Vous n'avez pas à modifier ce fichier.** Il est cependant nécessaire de comprendre cette classe afin de faire interagir vos agents avec l'état du jeu à l'instant t . Voici pour l'occasion une brève description du contenu de ce fichier pour vous aider à mieux l'appréhender :
 - `class GameStateQuoridor` : contient toutes les informations relatives à un état du jeu. Cette classe contient :

1. Un constructeur :

```
1 def __init__(self, scores: Dict[int, float],
2               active_player: Player,
3               players: List[Player],
4               rep: BoardQuoridor, step: int) -> None:
```

2. Diverses fonctions utilitaires :

```
1 def get_step(self)
2 def is_done(self)
3 def generate_possible_stateless_actions(self)
4 def generate_possible_stateful_actions(self)
5 def apply_action(self, action: StatelessAction)
```

- `my_player.py` : votre agent héritant de la classe `PlayerQuoridor`. **Vous devez modifier ce fichier.** Si vous souhaitez modifier des classes provenant des autres fichiers du jeu, il est recommandé d'utiliser l'héritage de Python dans ce fichier. N'oubliez pas de convertir les objets lorsqu'ils vous sont fournis (e.g. `GameStateQuoridor` lors d'un appel de `compute_action`)
- `random_player_quoridor.py` : un agent aléatoire si vous voulez gagner une partie facilement.
- `greedy_player_quoridor.py` : un agent glouton qui devrait perdre facilement face à vos agents ayant une profondeur de recherche plus élevée.

Le projet s'appuie sur le package open source `seahorse`. Vous pouvez le soutenir en déposant une étoile sur sa page GitHub et sa documentation. Pour lancer une partie il faut donc au préalable installer `seahorse` et les autres packages nécessaires à l'aide de la commande suivante :

```
$ pip install -r requirements.txt
```

Ensuite, plusieurs modes d'exécution sont disponibles via la présence d'arguments. Par exemple, `-r` permet d'enregistrer une partie dans un fichier json. Pour obtenir la description de tous les arguments, exécutez la commande suivante :

```
$ python main_quoridor.py -h
```

Pour lancer une partie en local avec GUI, il faut par exemple lancer la commande suivante :

INF8175	Projet - Agent intelligent pour le jeu Quoridor	Dest : Étudiants
Automne 2026		Auteurs: HB,EJ,GR

```
$ python main_quoridor.py -t local random_player_quoridor.py random_player_quoridor.py
```

La commande suivante lance une partie en local entre deux agent aléatoires, les logs sont enregistrés dans une json, la GUI n'est pas ouverte.

```
$ python main_quoridor -r -g -t local random_player_quoridor.py random_player_quoridor.py
```

Affronter un autre agent sur le réseau Il est possible d'organiser une partie contre un agent en réseau, sans devoir échanger le code source. Pour cela, il faut lancer une partie en host en premier lieu. Spécifiez votre adresse IP. Si vous n'êtes pas sur le même réseau local, vérifiez que les ports sont bien ouverts sur votre routeur afin de permettre la connexion à distance. Enfin il vous faudra également donner le nom de l'agent à distance (sans l'extension .py). Vous pouvez prendre en exemple la commande suivante pour héberger un match entre deux agents aléatoires :

```
$ python main_quoridor.py -t host_game -a <ip_address> random_player_quoridor.py "random_player_quoridor"
```

L'équipe que l'on souhaite affronter devra quant à elle lancer :

```
$ python main_quoridor.py -t connect -a <ip_address> random_player_quoridor.py
```

Il faudra alors remplacer <ip_address> par l'adresse IP de l'ordinateur qui héberge la partie. Pour obtenir cette dernière, exécuter la commande ipconfig (Windows) ou ifconfig (Mac, Linux) dans un terminal. Si le port de connexion par défaut (e.g. 16001) est utilisé, vous pouvez indiquer un nouveau en ajoutant le paramètre -p <port> dans la commande où <port> représente le numéro de port.

Partie contre un humain Afin de se familiariser au jeu dans un premier temps, il est possible de jouer manuellement l'un contre l'autre avec la commande suivante :

```
$ python main_quoridor.py -t human_vs_human
```

Finalement, pour pouvoir étudier le comportement de votre agent, il peut être intéressant de jouer manuellement contre ce dernier avec :

```
$ python main_quoridor.py -t human_vs_computer random_player_quoridor.py
```

En cas de problème, n'hésitez pas à communiquer avec votre chargé de laboratoire via Discord.

⚠ Il est préférable de ne pas utiliser le navigateur safari pour afficher l'interface graphique.

⚠ Nous vous conseillons vivement de vous assurer que vous êtes en mesure de faire tourner le code du projet au plus tôt.

4 Environnement virtuel

Il est important, pour la correction, de pouvoir reproduire aisément l'environnement dans lequel vous travaillez. Il en va de même pour le travail collaboratif et surtout pour l'organisation du tournoi. Maîtriser la gestion des environnements virtuels est un atout important pour votre future carrière.

⚠ L'ensemble des commandes suivantes doit être exécuté depuis la console dans le répertoire Quoridor.

1. Création d'un environnement virtuel

INF8175	Projet - Agent intelligent pour le jeu Quoridor	Dest : Étudiants
Automne 2026		Auteurs: HB,EJ,GR

```
$ python -m venv venv
```

Un dossier `venv` est normalement créé.

2. Activation de l'environnement virtuel

Pour Windows :

```
$ . .\venv\Scripts\Activate.ps1
```

Pour Linux et MacOS :

```
$ source venv/bin/activate
```

*La console devrait afficher (**venv**) comme préfixe.*

3. Construction des du fichier *requirements*

Le projet étant assez libre, il est possible (mais non obligatoire) d'ajouter des bibliothèques telles que numpy, par exemple. Il est donc nécessaire de fournir un fichier **requirements.txt** conformément au formalisme de ce type de fichier, afin que l'on puisse installer votre agent. L'environnement virtuel et pip permettent d'exporter simplement ces dépendances à l'aide de la commande suivante.

```
(venv) pip freeze > requirements.txt
```

*Attention, il faut veiller à exécuter cette commande avec l'environnement activé (présence du préfixe entre parenthèses). Il suffit maintenant de fournir le fichier **requirements.txt** dans le rendu ZIP.*

4. Vérification finale

Afin de s'assurer que l'export nous permettra de faire rouler votre agent lors du tournoi, il est utile de créer un nouvel environnement, d'y installer les dépendances exportées puis d'y faire jouer son agent.

```
$ python -m venv tmp
$ . .\tmp\Scripts\Activate.ps1    #(Windows)
$ source tmp/bin/activate         #(MacOS et Linux)
(tmp) pip install -r requirements.txt
```

*Si vous êtes capable de faire jouer une partie entre votre agent et un autre agent via cet environnement **tmp**, vous êtes assurés que nous pourrons installer et utiliser votre agent lors du tournoi et de la notation.*

5 Evaluation continue sur Abyss

Abyss est une plateforme conçue par l'équipe encadrante pour vous offrir un terrain de test où vous pouvez soumettre et évaluer la qualité de vos agents tout au long de la session. Chaque étudiant se voit attribuer un compte personnel doté d'un identifiant et d'un mot de passe, ce qui lui permet d'accéder à son profil pour y téléverser des agents. Le système fonctionne en continu, **permettant aux agents de s'affronter entre eux de manière automatique**, et ainsi, de consulter leurs performances. De plus, cela vous permettra de vous assurer que votre agent respecte toutes les consignes de soumission et d'utiliser les résultats de vos parties dans votre rapport de projet. **La soumission régulière d'agents sur Abyss constitue une partie de l'évaluation du projet** (voir les critères d'évaluation indiqués plus bas).

Le système qui mettra en place vos affrontements tourne sur un processeur **AMD Epyc 7371**. Les restrictions d'équité décrites dans la section précédente s'appliquent également. Le site de cette session est disponible sur Moodle. Si vous êtes curieux.se.s, une archive d'une session précédente sur le jeu Divercité est dispo-

INF8175	Projet - Agent intelligent pour le jeu Quoridor	Dest : Étudiants
Automne 2026		Auteurs: HB,EJ,GR

nible [ici](#). Vous pourrez utiliser cette version pour vous familiariser avec la structure de l'application.

Mise en ligne d'un agent. Une fois connecté, vous pourrez mettre en ligne votre agent depuis la page principale d'Abyss. Le format requis de mise en ligne est celui décrit à la section 7 du sujet. Il est primordial de suivre ce format, sans quoi votre agent ne pourra pas être lancé par le système.

Profil d'utilisateur. Votre profil utilisateur donne accès aux principales fonctionnalités d'interaction avec le système Abyss. Il est partagé avec votre binôme pour mettre en commun vos agents mis en ligne et les parties que ces derniers ont jouées. Après la mise en ligne d'un agent, ce dernier est mis en attente d'une validation avant de pouvoir jouer. Une fois la validation effectuée, si tout est en ordre, votre agent passera en état inactif, vous pourrez ainsi le changer afin de le faire entrer dans la compétition contre ses adversaires. Pour maintenir un équilibre et une compétition saine, chaque groupe d'étudiants est limité à **deux agents actifs en même temps**. Vous aurez également accès à une liste des parties effectuées par vos agents. Chaque étudiant a la possibilité de télécharger les données de ses anciennes parties au format **JSON**. Cette fonctionnalité est particulièrement utile pour les groupes qui souhaitent réanalyser leurs parties ou récolter des données pour la conception de leurs agents.

Classement Elo et le fonctionnement des ligues. Le classement Elo de chaque agent est un élément important du site Abyss. Ce classement détermine la ligue à laquelle un groupe appartient. Le classement Elo du groupe est défini par l'Elo maximum parmi les agents valides du groupe.

Visualisation des parties. Le site propose également une vue détaillée des parties du système, vous permettant de consulter les résultats de toutes les rencontres précédentes, mais aussi de voir quelles sont les parties en cours, ainsi que celles programmées et annulées. Les parties sont réparties dans les quatre sections selon leur statut. De plus, grâce à une interface graphique, vous pouvez revoir toutes les parties terminées afin de faciliter l'analyse des performances.

Page palmarès et statistiques. Une page dédiée au palmarès permet de visualiser les meilleures statistiques des groupes sur la plateforme. Vous pouvez consulter les groupes ayant obtenu les meilleures performances sur de nombreux critères, dans le but de favoriser une compétition saine et stimulante entre les élèves.

⚠ Si vous avez des questions concernant Abyss, n'hésitez pas à demander à vos chargés de laboratoire. De plus, si vous rencontrez un bogue ou une anomalie, utilisez le canal approprié du serveur Discord pour nous la signaler. Abyss est encore à un stade de développement récent. Si un problème survient et qu'une maintenance est nécessaire, les agents téléversés ne seront plus en train de jouer

Il est toujours possible, et fortement encouragé, de continuer de votre côté, de faire jouer vos agents, voire d'organiser d'autres affrontements avec les autres groupes. Nous vous invitons à partager avec nous vos commentaires et suggestions pour améliorer ce système sur notre serveur Discord ou via les évaluations sur Moodle.

6 Concours

Vous aurez également l'occasion de participer à un concours sous la forme d'un tournoi [Challenge](#). Le tournoi, facultatif, aura lieu vers la fin de la session, avec la version finale de votre agent. Le tournoi sera divisé

INF8175	Projet - Agent intelligent pour le jeu Quoridor	Dest : Étudiants
Automne 2026		Auteurs: HB,EJ,GR

en deux phases.

1. **Phase de qualification** : Les différents agents seront répartis en plusieurs poules. Tous les joueurs de la poule se rencontrent et chaque match se compose de 3 parties. À l'issue, les joueurs sont classés selon le nombre de victoires (et le cumul des coups nécessaires à la victoire en cas d'égalité). Seule une partie des meilleurs joueurs de chaque poule sera qualifiée pour la phase suivante.
2. **Phase éliminatoire** : Les joueurs s'affrontent les uns contre les autres en élimination directe jusqu'à la finale et la petite finale.

Sur les trois manches d'une partie, la couleur blanc/noir sera attribuée une fois à chaque joueur. La répartition de la dernière manche est aléatoire. Vous pourrez suivre les résultats de vos agents en direct sur le site du tournoi. Le tournoi aura lieu quelques jours après la remise de la version compétitive de votre agent. Le lien du tournoi ainsi que sa configuration finale vous seront fournis ultérieurement. Les matchs seront lancés sur la même machine que celle qui héberge Abyss, les spécifications sont donc les mêmes que celles décrites dans la section 5.

7 Code à produire

Vous devez remettre un fichier `my_player.py` avec tous les fichiers complémentaires de votre projet. Pour ce faire, vous êtes entièrement libres d'utiliser la méthode de votre choix. Si vous utilisez des bibliothèques externes (telles que `numpy`, `keras`, `pytorch`, etc.), veuillez les lister dans le fichier `requirements.txt` lors de votre soumission, comme décrit dans la section 4. **Excluant `my_player.py`, les autres fichiers fournis ne doivent pas être ni modifiés ni rendus à nouveau.** Veuillez vous référer à la section 3 pour plus d'indications si vous souhaitez modifier les autres classes fournies.

Au minimum, votre solution devra hériter de la classe `PlayerQuoridor` et implémenter la fonction centrale `compute_action()`. Cette fonction doit retourner une action selon l'état actuel du jeu :

Input : `self:PlayerQuoridor, current_state:GameStateQuoridor`.

Toutes les informations de la partie (placement des murs, position des pions, murs restants de chaque joueur) sont contenues dans l'objet `current_state`. Les autres informations présentes (nombre de coups joués, temps restant, etc.) peuvent également aider votre agent à prendre des décisions durant la partie en se repérant mieux dans la chronologie et dans l'état de son avancement.

Output : `Action`

Une action est un dictionnaire contenant deux clés

- `type` : le type d'action (`move` pour un déplacement, `vertical` ou `horizontal` pour placer un mur dans cette orientation).
- `destination` : les coordonnées de la case concernée par l'action. Ici, la destination est un tuple de $[0,8]^2$. Pour un déplacement, il s'agit de la case de destination. Pour un mur vertical, il s'agit de la case située au sud-est (en bas à droite) du point le plus au nord du mur. Pour un mur horizontal, c'est la case située au sud-est du point le plus à l'ouest du mur. Par exemple, dans la Figure 6, le mur séparant les cases b6/b7 des cases c6/c7 est un mur vertical posé en c6 (donc en (2,5)). Le mur séparant les cases d4/e4 des cases d5/e5 et un mur horizontal placé en d5 (donc en (3,4)).

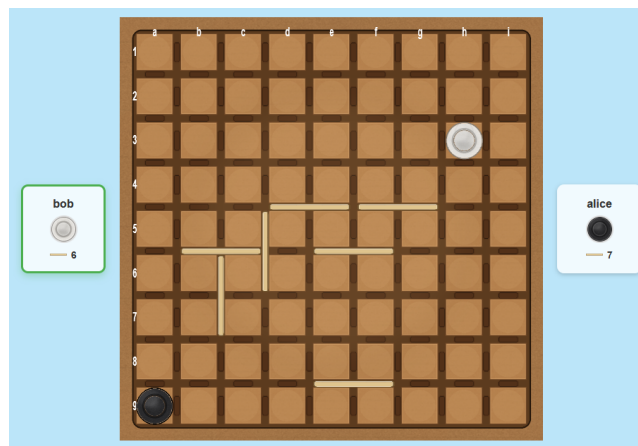


FIGURE 6 – Exemple de plateau

Pour obtenir un exemple minimal d'un joueur fonctionnel (mais insuffisant pour répondre aux attentes du projet), vous pouvez consulter les agents `random_player_quoridor.py` et `greedy_player_quoridor.py`.

⚠ **Dès lors que l'agent soumet une action, il doit s'assurer qu'elle est bien valide (c.-à-d. qu'elle figure dans l'ensemble retourné par `current_state.get_possible_stateless_actions()`).** Si une action non autorisée est soumise, le joueur perd automatiquement la partie au profit de son adversaire. Un vérificateur de validité des actions est utilisé dans `seahorse` et lève une exception de type `ActionNotPermittedError`.

7.1 Instructions de remises supplémentaires

Le format décrit dans ces instructions est celui à privilégier pour tous vos rendus du projet. Il est également obligatoire pour les soumissions sur Abyss et lors du concours.

Lors du concours, il est nécessaire de nommer le zip de rendu du code au format suivant :

NomEquipe_matricule1_matricule2.zip.

Vous êtes libre de choisir votre nom d'équipe (mais il doit rester acceptable), sinon nous vous attribuerons votre numéro d'équipe sur Moodle.

Pour Abyss, le nom de votre fichier zip n'a pas d'importance et ne sera pas pris en compte.

- Pour les bibliothèques, assurez-vous de **ne fournir que celles nécessaires à l'exécution** de votre agent. Les bibliothèques d'entraînement inutilisées par l'agent final ne doivent pas être fournies.
- Si vous souhaitez ajouter des **fichiers supplémentaires** : mettez-les dans un dossier nommé au format *src_matricule1_matricule2*. Assurez-vous que les liens (les `import`) entre les fichiers sont corrects et que votre agent fonctionne avec cette architecture. Ce format est privilégié afin d'éviter d'écraser les fichiers de deux agents différents lors d'un match.
- Pour tout fichier supplémentaire, veuillez également utiliser des noms de fichiers sans espaces, par exemple, `my_function.py` au lieu de `my fonction.py`. Les accents et les caractères spéciaux autres que le tiret bas sont également à éviter.

Un point important à prendre en compte lors de la soumission d'un agent sur le système Abyss est que les fichiers JSON peuvent contenir des informations à leur sujet ; il s'agit des attributs de votre classe héritant de `PlayerQuoridor` et des valeurs qu'ils possèdent à la fin de votre tour de jeu. Si vous ne souhaitez pas que ces attributs apparaissent, il vous suffit de les **renommer en les préfixant d'un tiret bas**. Vous pourrez retrouver un exemple ci-dessous :

```

1 class MyPlayer(PlayerQuoridor):
2     def __init__(self, piece_type: str, name: str = "bob",*args) -> None:
3         super().__init__(piece_type,name,*args)
4         self.count_pieces = 0 # Cet attribut apparaîtra dans le JSON
5         self._nodes = [] # Cet attribut n'apparaîtra pas dans le JSON
6     def compute_action(self, current_state: GameState, **kwargs) -> Action:
7         possible_actions = current_state.get_possible_stateless_actions()
8         # Variable non contenu comme attribut, elle n'apparaîtra pas dans le JSON
9         ...

```

Nous vous recommandons d'utiliser ce motif de noms afin de filtrer les enregistrements de vos parties et les attributs que vous ne souhaitez pas voir apparaître. N'oubliez pas que vos adversaires ont aussi accès à ce fichier;) Mais il est également recommandé de filtrer les attributs susceptibles de référencer des structures de données pouvant grandir au cours de votre tour de jeu et/ou contenant beaucoup de données. Par exemple, une liste de nœuds d'un MCTS ou une table de transpositions devraient être filtrées s'ils prennent trop de valeurs. La raison de cette recommandation est la limitation de l'espace mémoire et de la capacité de calcul du serveur d'Abyss, mais également le risque de crash lors de l'écriture qui entraînerait la perte du fichier.

8 Critères d'évaluation

L'évaluation du projet (40 points) est ventilée de la façon suivante :

- 20 points pour la qualité de votre agent.
- 10 points pour un rapport de votre conception solution.
- 10 points pour le travail continu le long de la session.

Ces trois critères sont détaillés ci-dessous.

Qualité de votre agent (20 points) Il s'agit de la qualité et de l'efficacité de votre agent face à 6 agents cachés d'une difficulté croissante. Parmi ces agents cachés, plusieurs ont été générés à l'aide d'intelligences artificielles. Ainsi, bien que l'utilisation d'IAs génératives soient permises dans le cadre du projet (voir les restrictions dans le plan de cours), une utilisation médiocre ne vous donnera pas beaucoup de points. Une victoire rapporte 2 points par agents, pour un total de 12 points. De plus, 6 points sont donnés pour la qualité de vos conceptions personnelles sur votre agent. Il peut s'agir de vos fonctions heuristiques, d'une utilisation intelligence du timer, d'une détection de symétries, etc. Veillez à ce que ces points soient bien illustrés dans le rapport. Finalement, les 2 points restants sont donnés pour la qualité de votre code (bonne structure, commentaires, etc.). Comme règle d'or, nous nous attendons à ce que le code soit clair et bien documenté, afin qu'il soit facile à comprendre pour les correcteurs.

Travail continu (10 points) Votre travail continu tout au long de la session est également évalué. Concrètement, il vous est demandé de soumettre un nouvel agent environ toutes les 2 semaines sur Abyss, pour un total de 5 versions (2 points par version). Le planning exact est indiqué au début de ce document. Il n'est pas attendu que toutes les versions soumises soient efficaces, mais elles doivent comporter une amélioration ou différence significative par rapport aux précédentes. Par exemple, changer le paramètre de votre heuristique n'est pas une amélioration significative. Par contre, ajouter plusieurs éléments nouveaux à votre heuristiques est accepté. Il est bien sûr possible (et encouragé) de soumettre plus de versions que ce qui est demandé. Ces points sont très facilement gagnés, et ont pour objectif de vous inciter à répartir votre travail

INF8175	Projet - Agent intelligent pour le jeu Quoridor	Dest : Étudiants
Automne 2026		Auteurs: HB,EJ,GR

tout au long de la session.

Rapport de votre conception (10 points) Vous devez rédiger un rapport qui détaille votre stratégie de recherche et vos choix de conception. Celui-ci peut être rédigé, au choix, *en français ou en anglais* et rendu au *format PDF*. Étant donné que vous êtes libres d'implémenter la solution de votre choix, qu'elle soit inspirée de concepts vus en cours ou de vos connaissances personnelles, nous attendons une explication claire et détaillée du fonctionnement de votre agent. N'écrivez pas d'introduction à votre rapport et ne répétez pas les informations données dans les slides (p.ex., n'expliquez pas comment fonctionne une stratégie mini-max). Vous pouvez présenter directement votre méthodologie. Précisément, votre rapport doit contenir au minimum les informations suivantes :

1. **Titre du projet.**
2. **Nom d'équipe** sur Challonge, ainsi que la liste des membres de l'équipe (nom complet et matricule).
3. **Méthodologie** : Explication du fonctionnement de votre agent, des choix de conception faits (fonctionnement de l'heuristique utilisée, stratégie choisie, spécificités propres à votre agent, gestion des 15 minutes allouées, etc.). Cette partie est fortement liée aux 6 points qui sont donnés pour les choix de conception de votre agent.
4. **Evolution de l'agent** : Indiquez clairement mais brièvement le cheminement de vos 5 versions, et comment chacune améliore (ou change) la précédente. Indiquez les noms que vous avez soumis sur Abyss. Cette partie est fortement liée aux 10 points qui sont donnés pour l'évaluation continue.
5. **Résultats** : Reportez les performances de votre agent en le testant contre les implémentations qui vous sont fournies ainsi que vos différentes versions. Spécifiez un ou plusieurs critères de mesure pour chiffrer vos résultats et rédiger une analyse. Au-delà du simple taux de victoire ou du score, mettez-les en avant à l'aide de graphiques, de tableaux ou de toute autre technique de visualisation pertinente selon vous. Par exemple, si votre agent utilise une méthode de pruning, intéressez-vous à montrer qu'elle est efficace et quels sont ses apports en termes de performances. Nous vous suggérons d'utiliser vos parties locales ainsi que celles sur Abyss pour étayer vos analyses. Il sera nécessaire de limiter les comparaisons avec les agents fournis (p. ex. aléatoire et greedy), car ils ne sont destinés qu'à vous aider à démarrer votre projet et sont très rudimentaires.
6. **Discussion** : Discutez des avantages et des limites de votre agent final. Évoquez différentes pistes d'amélioration, selon vous.
7. **Références (si applicable).**

Le rapport ne doit pas dépasser **6 pages** et doit être rédigé sur une ou deux colonnes à simple interligne, avec une police de caractères de taille 10 ou plus (des pages supplémentaires pour les références et le contenu bibliographiques sont autorisées, ainsi que pour la page de garde). Aucune autre annexe n'est autorisée. Vous êtes libre de structurer le rapport comme vous le souhaitez.

⚠ **La note minimale sera attribuée à quiconque ayant copié/collé une solution ou proposant un code contenant une erreur de compilation. Prenez bien soin de vérifier le contenu de votre rendu. Notez que, vu la liberté laissée à l'implémentation, il est très facile de détecter les cas de plagiat.**

⚠ **L'évaluation sera effectuée sur une machine Linux raw, veuillez donc à bien spécifier (si nécessaire) les bibliothèques externes dans un fichier `requirements.txt` afin de pouvoir les installer avec la commande `pip install -r requirements.txt`.**

⚠ **Notez bien que le classement de votre agent durant le tournoi n'aura aucune influence sur l'évaluation. Dès lors, il est tout à fait possible d'être éliminé rapidement lors du concours et d'obtenir une note maximale, ou même d'être premier au concours et d'obtenir une mauvaise évaluation (p.ex., à cause d'un rapport négligé). Nous nous attendons à ce que le code soit clair et bien documenté, afin qu'il soit facile à comprendre pour les correcteurs.**

INF8175	Projet - Agent intelligent pour le jeu Quoridor	Dest : Étudiants
Automne 2026		Auteurs: HB,EJ,GR

⚠ Le jury préférera une solution simple accompagnée d'une riche explication logique à une solution trop complexe mettant en œuvre des concepts mal compris de l'élève.

9 Quelques conseils pour vous aider

Conseils généraux

- Assurez-vous de bien comprendre le fonctionnement théorique des méthodes que vous souhaitez utiliser avant de les implémenter. Également, il est plus que conseillé de réaliser d'abord les agents du Morpion et de Puissance 4 du deuxième laboratoire.
- Prenez le temps de comprendre la structure du code de `game_state_quoridor.py` en y ajoutant des "*print statements*" et des commentaires. Si cela peut vous sembler laborieux, vous gagnerez beaucoup de temps par la suite lors de vos éventuels *debuggings* (qui arriveront quasi certainement :-)).
- Commencez par créer des agents simples, puis ajoutez des fonctionnalités progressivement.
- Trouvez le bon compromis entre complexité et facilité d'implémentation lors de la conception de votre agent. Le temps de calcul mis à disposition de votre agent pour chaque action étant limité, il doit également être pris en compte.
- Travaillez efficacement en groupe et répartissez bien les tâches.
- Tirez le meilleur parti des séances de laboratoire pour demander des conseils.
- Profitez de ce travail de groupe pour vous initier à l'outil de développement collaboratif Git ainsi qu'aux bonnes pratiques qui y sont associées.
- Un agent efficace n'est pas forcément celui qui utilise les méthodes les plus complexes. Un algorithme simple mais bien implémenté, robuste et efficace a toutes ses chances.
- N'attendez pas d'avoir un agent parfait avant de le soumettre sur Abyss. Vous recevrez ainsi des rétroactions intermédiaires qui permettront d'améliorer itérativement vos agents.

Suggestion d'architecture pour votre agent En fonction de vos connaissances préalables et de vos intérêts respectifs, nous vous proposons plusieurs options d'architecture pour la conception de vos agents.

- *Niveau standard* : Procédez avec un algorithme de type *minimax* doté d'une bonne heuristique et enrichi de mécanismes supplémentaires. Ce type d'algorithme a gagné le concours en automne 2021, 2022, 2023, 2025 et 2026. C'est un algorithme de ce type que nous vous recommandons pour le projet.
- *Niveau avancé* : Procédez avec un algorithme de type *Monte Carlo Tree Search*. Il peut être intéressant de chercher des ressources complémentaires pour vous aider, par exemple des articles détaillant des architectures pour d'autres jeux. Plusieurs algorithmes de ce type figuraient dans le top 10 des agents en automne 2021 et 2022.
- *Niveau expert* : Procédez avec un algorithme d'*apprentissage automatique*. Aucun algorithme de ce type n'a bien performé lors des éditions précédentes et demande généralement beaucoup de travail. Si vous décidez de vous y tenir, il est plus que conseillé de consulter de la documentation supplémentaire à ce sujet.

⚠ **Partir sur une architecture plus complexe n'a aucun impact sur la note que vous recevrez et rendra votre projet plus compliqué. Les niveaux avancés sont davantage destinés aux étudiants souhaitant expérimenter des algorithmes de leur choix.**

Bon travail, bonne chance, et surtout, amusez-vous!

